

UNIVERSIDAD AUTÓNOMA DE ZACATECAS

"FRANCISCO GARCÍA SALINAS"



UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA
INGENIERÍA DE SOFTWARE 4° D



Proyecto: FashionFlow POS

Materia: Análisis y diseño orientado a objetos

Docente: Dr. Cristian Boyain

Integrante 1: Julio Miguel Sifuentes Sánchez

Integrante 2: Ana Patricia Galven Cruz

Versión: 5.0

Fecha: 04/06/2026

Introducción

1.1 Propósito del Documento

El presente documento constituye la Especificación de Requisitos de Software (SRS) para el sistema **FashionFlow POS**, desarrollado como proyecto integrador de la materia Análisis y Diseño Orientado a Objetos. Su propósito es definir de manera formal y verificable los requisitos funcionales y no funcionales del sistema, sirviendo como referencia para el diseño, desarrollo y validación del producto.

1.2 Alcance del Sistema

FashionFlow POS es un sistema de punto de venta orientado a tiendas de ropa y calzado. El sistema cubre los procesos de venta, gestión de inventario, administración de catálogo de productos, organización por departamentos y generación de reportes de actividad comercial. Queda fuera del alcance la integración con plataformas de comercio electrónico y la gestión de nómina del personal.

1.3 Definiciones, Acrónimos y Abreviaturas

Término	Definición
POS	Point of Sale — Sistema de Punto de Venta.
SRS	Software Requirements Specification — Especificación de Requisitos de Software.

SKU	Stock Keeping Unit — Unidad de mantenimiento de inventario.
CRUD	Create, Read, Update, Delete — Operaciones básicas de datos.
MVC	Model-View-Controller — Patrón de arquitectura de software.
CU	Caso de Uso.
EBP	Elementary Business Process — Proceso Empresarial Elemental.
OOP	Object-Oriented Programming — Programación Orientada a Objetos.

1.4 Referencias

- Larman, C. (2004). *Applying UML and Patterns*, 3rd Edition. Prentice Hall.
- Material de clase: Guías de Casos de Uso — Dr. Cristian Boyain, UAZ, 2026.
- IEEE Std 830-1998: IEEE Recommended Practice for SRS.

1.5 Visión General del Documento

El documento se organiza en ocho secciones: Introducción, Descripción General, Requisitos de Interfaz, Requisitos Funcionales y No Funcionales, Diseño Orientado a Objetos, Casos de Uso, Arquitectura del Sistema y Conclusión.

2. Descripción General del Sistema

2.1 Perspectiva del Producto

FashionFlow POS es un sistema independiente de escritorio diseñado para operar en entornos de tienda física. No depende de sistemas externos para su operación básica; sin embargo, puede integrarse con servicios de autorización de pagos mediante interfaces definidas. El sistema está optimizado para ser usado por cajeros con entrenamiento básico y por administradores del negocio.

2.2 Funciones Principales del Producto

Función	Descripción Breve
Procesamiento de Ventas	Registro y cierre de transacciones comerciales en caja.
Gestión de Inventario	Consulta y control de existencias físicas por artículo.
Administración de Productos	Altas, modificaciones y bajas del catálogo.
Administración de Departamentos	Organización jerárquica de categorías comerciales.
Análisis de Actividad Comercial	Generación de reportes de rendimiento del negocio.

2.3 Características de los Usuarios

Tipo de Usuario	Nivel Técnico	Responsabilidades Principales
Cajero	Básico	Procesar ventas y consultar inventario en caja.
Administrador del Sistema	Intermedio	Gestionar catálogo, departamentos y reportes.

2.4 Restricciones Generales

- El sistema opera bajo el patrón de arquitectura MVC.
- La jerarquía de clases de productos debe respetar los principios SOLID.
- Los casos de uso se especifican en estilo esencial (sin UI) y formato caja negra, conforme a las guías del Dr. Boyain.
- El sistema no gestiona la autorización de pagos directamente; es responsabilidad de un actor externo.

2.5 Suposiciones y Dependencias

- Se asume que cada terminal POS dispone de conexión a la base de datos local.
- El personal cajero recibe capacitación mínima antes de operar el sistema.
- Los servicios de autorización de pago externos son provistos por terceros.

3. Requisitos de Interfaz

3.1 Interfaces de Usuario

El sistema expondrá interfaces funcionales alineadas con los módulos principales: pantalla de ventas, pantalla de inventario, pantalla de administración de catálogo y pantalla de reportes. Las interfaces serán diseñadas para flujo de trabajo ágil orientado al cajero con entrenamiento.

Nota: El diseño detallado de la UI se definirá en la fase de diseño de interfaz; los casos de uso de este documento se redactan en estilo esencial sin UI.

3.2 Interfaces de Hardware

- Lector de código de barras compatible con entrada USB/HID para captura de SKU.
- Impresora de tickets térmica compatible con protocolos ESC/POS.
- Terminal de pago externo (responsabilidad del proveedor de pagos).

3.3 Interfaces de Software

- Sistema operativo: Windows 10 / Linux Ubuntu 22.04 o superior.
- Motor de base de datos relacional local (abstracción mediante capa de repositorio).
- API REST para comunicación con el módulo remoto de análisis de actividad de ventas.

3.4 Interfaces de Comunicación

- Protocolo HTTPS para la transmisión segura de datos de ventas al sistema de análisis remoto.
- Formato JSON para el intercambio de datos entre nodos POS y el servidor central.

4. Requisitos Funcionales y No Funcionales

4.1 Requisitos Funcionales

Los siguientes requisitos funcionales describen el comportamiento observable del sistema (qué debe hacer), en concordancia con los casos de uso definidos en la Sección 6.

ID	Requisito Funcional	CU Relacionado
RF-01	El sistema debe registrar transacciones de venta identificando productos por SKU, calculando subtotales, impuestos y total.	CU-01

RF-02	El sistema debe deducir las existencias de los productos involucrados al confirmar una venta.	CU-01
RF-03	El sistema debe permitir la consulta de existencias por SKU o por departamento.	CU-02
RF-04	El sistema debe procesar la incorporación, actualización y baja de artículos del catálogo, respetando la jerarquía de herencia.	CU-03
RF-05	El sistema debe gestionar el alta y edición de categorías comerciales (departamentos) con asignación jerárquica de productos.	CU-04
RF-06	El sistema debe recopilar registros de operaciones, calcular métricas y generar reportes de rendimiento del periodo solicitado.	CU-05

4.2 Requisitos No Funcionales

4.2.1 Rendimiento

- El tiempo de respuesta para el registro de una venta no debe exceder los 2 segundos bajo carga normal.
- La consulta de inventario debe retornar resultados en menos de 1 segundo para catálogos de hasta 10,000 artículos.

4.2.2 Confiabilidad

- El sistema debe garantizar consistencia de datos en caso de fallo durante una transacción (atomicidad).
- La disponibilidad del sistema en horario de operación debe ser igual o superior al 99%.

4.2.3 Mantenibilidad

- La arquitectura MVC y los principios SOLID deben facilitar la incorporación de nuevos tipos de productos sin modificar clases base existentes.
- El código debe estar documentado con comentarios de propósito en métodos públicos.

4.2.4 Seguridad

- El acceso al sistema debe requerir identificación y autenticación del usuario antes de habilitar cualquier funcionalidad.
- Las operaciones de administración estarán restringidas al rol Administrador.

4.2.5 Usabilidad

- Un cajero con entrenamiento básico debe ser capaz de procesar una venta estándar en menos de 3 minutos.
- El sistema debe mostrar mensajes de error claros y descriptivos ante operaciones inválidas.

5. Diseño Orientado a Objetos

5.1 Jerarquía de Herencia — Productos

La clase abstracta **Producto** centraliza los atributos y métodos comunes. Cada departamento es una subclase que hereda de ella y puede extender sus propiedades según sus necesidades específicas.

Clase Base: Producto

- Atributos: sku: String, nombre: String, precio: Float, existencias: Int, departamento: String.
- Métodos: obtener_info(): String, aplicar_descuento(p: Float): Float.

Clases derivadas por herencia:

Clase	Extensión
1. Calzado	Añade talla: String, material: String.
■ CalzadoDama	Hereda de Calzado, añade tacon: Bool.
■ CalzadoCaballero	Hereda de Calzado, añade tipo: String.
2. RopaDeportiva	Añade deporte: String, genero: String.
3. VestidoDeNoche	Añade ocasion: String, tela: String.
4. RopaHombre	Añade talla: String.
5. RopaMujer	Añade talla: String.
6. Accesorio	Añade tipo_accesorio: String.

5.2 Diagrama de Clases Principal

Estructura y métodos base de las principales clases del sistema:

Clase	Atributos	Métodos
Producto	sku, nombre, precio, stock	info(), descuento()
Inventario	productos[], umbral	agregar(), reducir(), buscar()
Venta	id, fecha, cajero, total	calcular(), cerrar()
DetalleVenta	producto, cantidad, precio	subtotal()

Clase	Atributos	Métodos
Departamento	nombre, productos[]	agregar(), listar()
Usuario	id, usuario, rol	autenticar()
Reporte	tipo, rango	generar(), exportar()

6. Casos de Uso

6.1 Actores y Límites del Sistema

Para el modelado, el software FashionFlow POS define el límite del sistema bajo diseño. Se identifican exclusivamente los actores externos que interactúan directamente con los servicios del sistema para obtener un resultado observable de valor comercial. El software en sí o sus automatizaciones no se consideran actores secundarios.

Actor	Tipo	Descripción de la Intención / Objetivo
Cajero	Primario	Satisface el objetivo de procesar las transacciones de los clientes en caja y verificar la disponibilidad inmediata de las prendas o calzado.
Administrador	Primario	Satisface los objetivos de gestión estructural del catálogo, la configuración de departamentos y el análisis estratégico del rendimiento del negocio.

6.2 Catálogo de Casos de Uso

Se eliminan los casos de uso técnicos o de interfaz (como Iniciar Sesión), ya que fallan la **Prueba del Jefe** al no representar una tarea de negocio por sí misma. Los requerimientos de datos se unifican bajo la convención idiomática 'Administrar X' y se nombran mandatoriamente iniciando con un verbo en infinitivo.

- **CU-01 Procesar Venta:** Registra las transacciones en el punto de venta agregando valor comercial directo (Pasa la Prueba de Proceso Empresarial Elemental — EBP).
- **CU-02 Consultar Inventario:** Permite comprobar las existencias físicas de los artículos comerciales de la tienda.
- **CU-03 Administrar Productos:** Centraliza los comportamientos de altas, modificaciones y bajas del catálogo de mercancías.
- **CU-04 Administrar Departamentos:** Gestiona la categorización de los nuevos departamentos de ropa y calzado sin alterar las clases base existentes.
- **CU-05 Analizar Actividad Comercial:** Evalúa el rendimiento financiero y consolidado del negocio dentro de un periodo determinado.

6.3 Especificación Esencial y de Caja Negra

Las siguientes especificaciones describen únicamente las intenciones de los actores y las responsabilidades del sistema, omitiendo detalles de pantallas (UI) o especificaciones de bases de datos internas (como sentencias SQL).

ID y Caso de Uso	Actor	Descripción del Flujo (Responsabilidades del Sistema)
CU-01 Procesar Venta	Cajero	Sistema identifica los productos seleccionados mediante su SKU. Sistema calcula los subtotales, impuestos y el importe total aplicando las reglas de descuento correspondientes. Al confirmarse la transacción, sistema registra la venta y deduce de forma consistente las existencias de los productos involucrados.
CU-02 Consultar Inventario	Cajero / Administrador	Sistema solicita los criterios de búsqueda (SKU o departamento). Sistema recupera y expone el estado de las existencias actuales correspondientes a la consulta.
CU-03 Administrar Productos	Administrador	Sistema procesa la incorporación, actualización de propiedades (precio, stock) o la inhabilitación de artículos del catálogo comercial, respetando las propiedades heredadas en la jerarquía de herencia.
CU-04 Administrar Departamentos	Administrador	Sistema procesa el alta o la edición de las categorías comerciales (ej. Calzado Caballero, Vestido Noche), permitiendo la asignación jerárquica de productos a sus respectivos módulos.
CU-05 Analizar Actividad Comercial	Administrador	Sistema recopila los registros de operaciones del periodo solicitado. Sistema calcula las métricas consolidadas y genera un reporte estructurado de rendimiento para su uso analítico.

7. Arquitectura del Sistema — Patrón MVC

El sistema utiliza el patrón **Modelo-Vista-Controlador (MVC)** para separar las responsabilidades de datos, interfaz e interacciones, facilitando la modularidad y el mantenimiento.

Capa	Responsabilidad	Componentes Principales
Modelo	Administra datos y lógica de negocio.	Producto, Inventario, Usuarios, Reportes.

Vista	Expone los elementos visuales esenciales al usuario.	Pantallas de Ventas, Inventario, Reportes y Control de Acceso.
Controlador	Conecta la Vista con el Modelo, procesando operaciones lógicas.	Registrar venta, Actualizar inventario, Generar reportes.

Principios SOLID Aplicados

S — Responsabilidad Única: Cada clase tiene una sola razón de cambio; la clase Venta únicamente gestiona el procesamiento de transacciones comerciales.

O — Abierto / Cerrado: Las nuevas variaciones de ropa o calzado se añaden como subclases directas sin modificar el comportamiento ni la estructura de la clase base Producto.

L — Sustitución de Liskov: Cualquier subclase derivada (CalzadoDama, RopaDeportiva, etc.) puede utilizarse indistintamente donde se requiera un objeto del tipo abstracto Producto.

D — Inversión de Dependencias: Los controladores interactúan con abstracciones y contratos de los productos, impidiendo la dependencia directa de implementaciones acopladas.

Nota: El principio I (Segregación de Interfaces) se aplica implícitamente al definir contratos separados para cada tipo de actor en los casos de uso.

8. Conclusión

FashionFlow POS es un sistema modular enfocado en optimizar el control de inventario y el proceso de venta de tiendas de calzado y textiles.

A través del uso estricto del análisis orientado a objetos, se ha desarrollado una estructura de herencia sólida que resuelve de manera limpia la diversidad de productos de la tienda.

La depuración de la especificación de requisitos hacia casos de uso esenciales y de caja negra garantiza que el sistema esté diseñado en función del valor real del negocio y no amarrado a interfaces de usuario o tecnologías particulares, previniendo errores de diseño e incrementando la mantenibilidad a largo plazo.

Este documento se consolida como la especificación formal que guiará el desarrollo de la arquitectura de software, la codificación de controladores y el diseño posterior de interfaces.